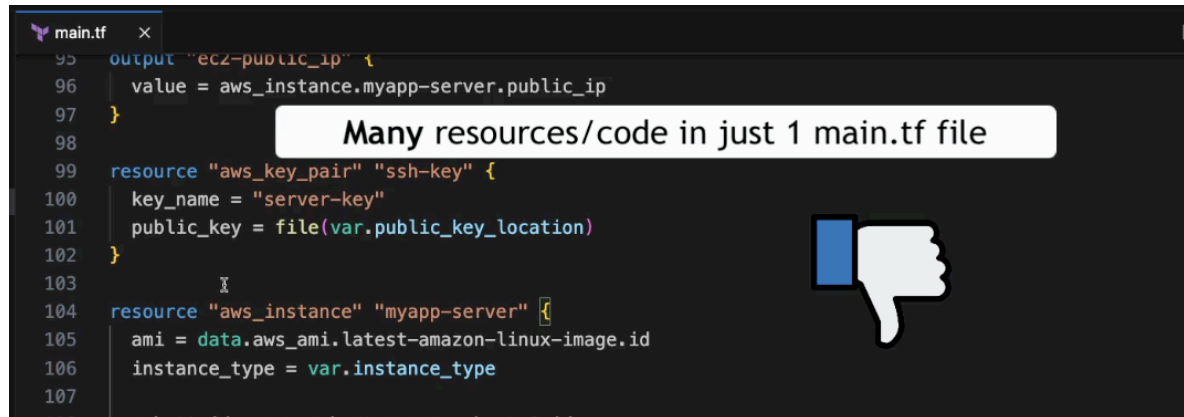


Module 12 - chapter 17

Modules in Terraform - Part 2

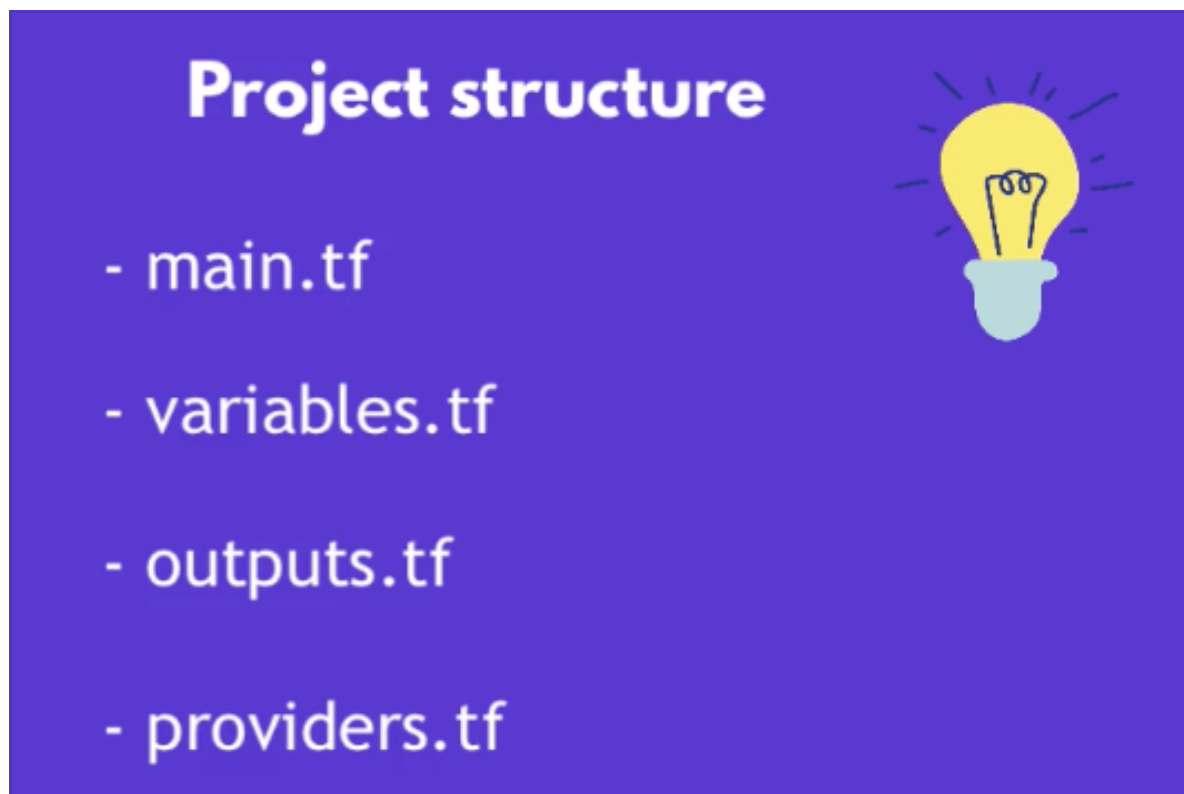
Modularise our project



The screenshot shows a code editor with a file named `main.tf`. The code contains Terraform configuration for an AWS EC2 instance and an SSH key pair. A white callout box with the text "Many resources/code in just 1 main.tf file" is overlaid on the code. To the right of the callout box is a thumbs down icon, indicating that having many resources in a single file is a negative practice.

```
95 output "ec2-public-ip" {
96   value = aws_instance.myapp-server.public_ip
97 }
98
99 resource "aws_key_pair" "ssh-key" {
100   key_name = "server-key"
101   public_key = file(var.public_key_location)
102 }
103
104 resource "aws_instance" "myapp-server" {
105   ami = data.aws_ami.latest-amazon-linux-image.id
106   instance_type = var.instance_type
107 }
```

Let's start by clearing the main.tf file by following a best practice TF structure:



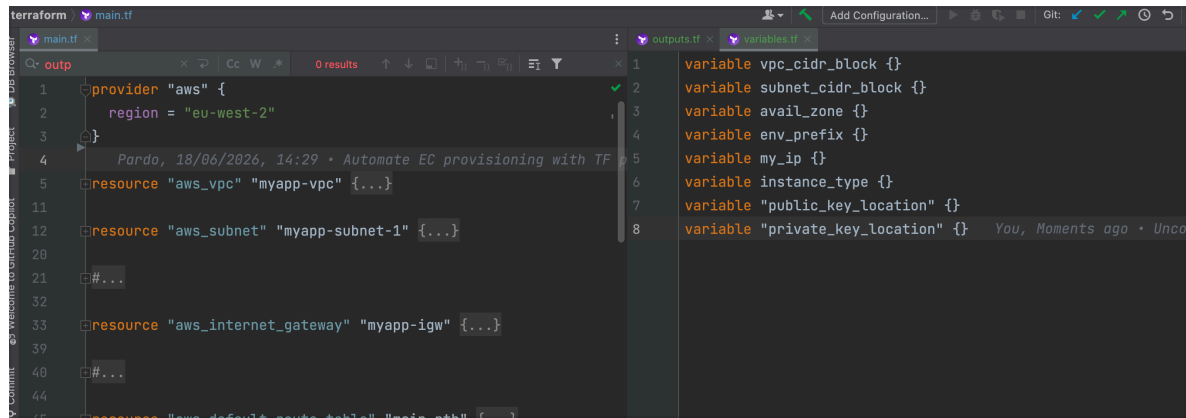
Project structure

- main.tf
- variables.tf
- outputs.tf
- providers.tf

A lightbulb icon is shown to the right of the list, indicating a good idea or best practice.

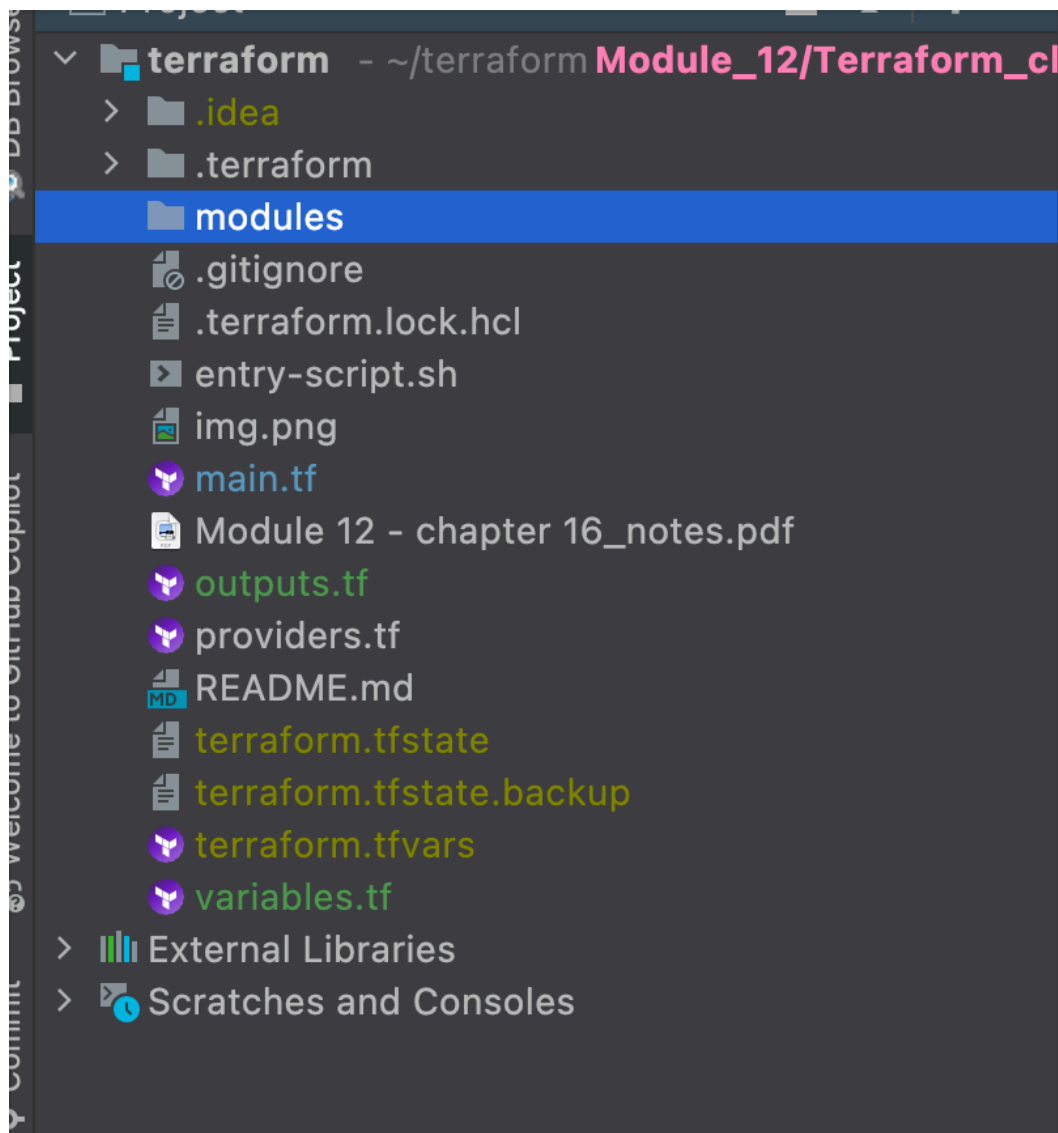
So we will move the variables out of main.tf and into its own file variables.tf, same for outputs (we already have the providers.tf)

So main.tf will only have resource definitions

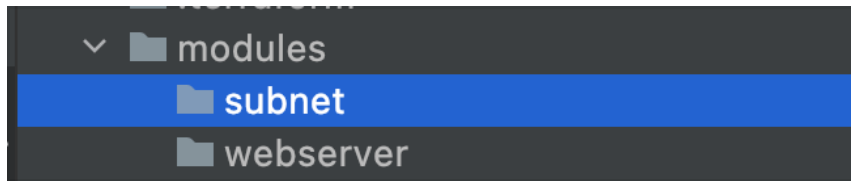


Create a Module

We create a new folder called -> 'modules'



And inside the folder 'modules' we will create folder for the actual modules. For example two new folders, one called "weber" and another called "subnet"



Each folder will have their own 📌

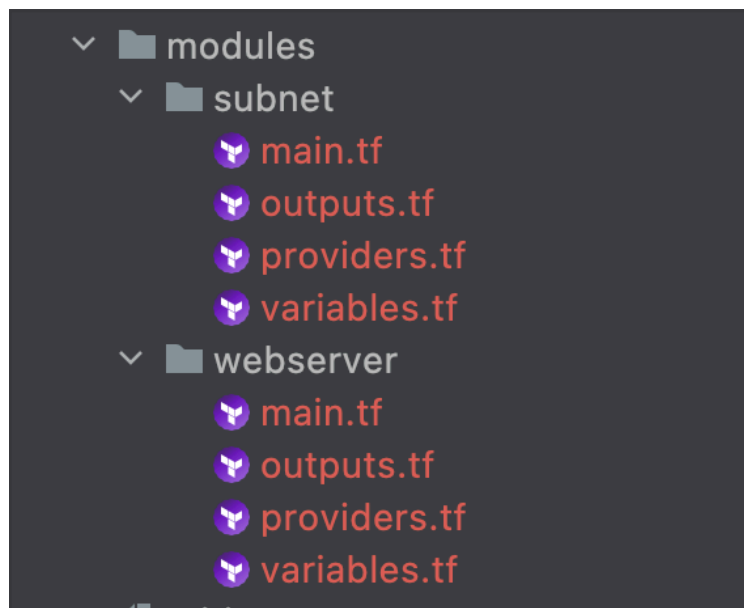
Project structure

- main.tf
- variables.tf
- outputs.tf
- providers.tf



We will create those files in each folder using the terminal.

```
> cd ..  
> cd subnet  
> touch main.tf  
> touch outputs.tf  
> touch variables.tf  
> touch providers.tf
```



So our project now has this structure

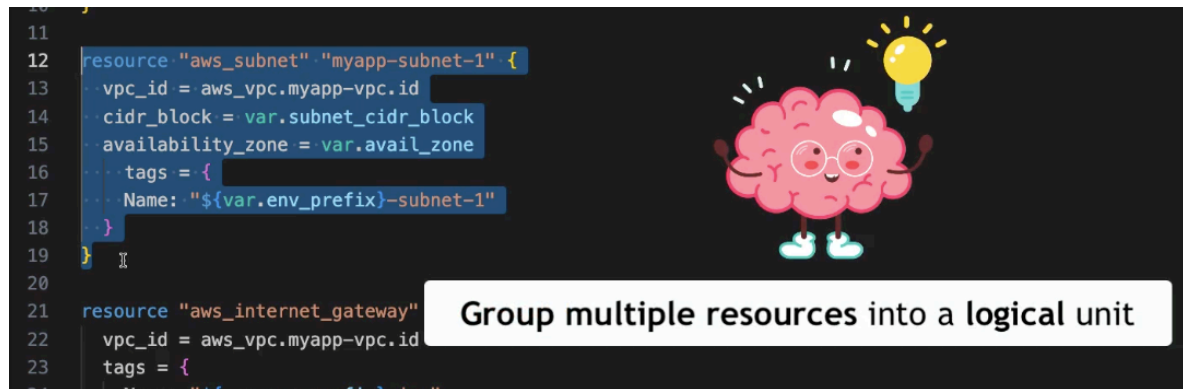
Project structure

- root module
- /modules = "child modules"

"child module" - a module that is called by another configuration

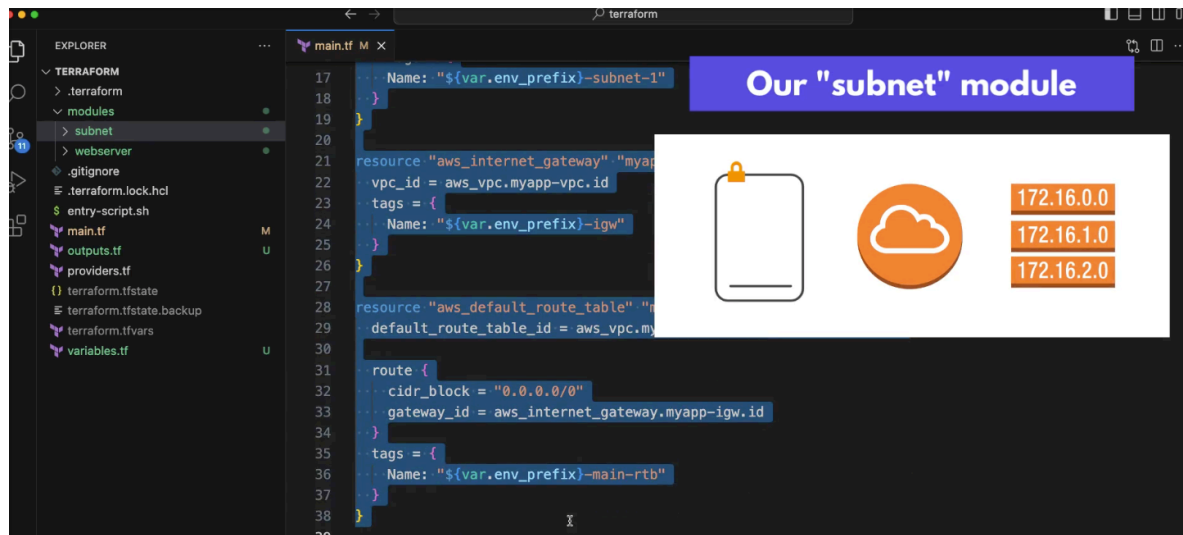
A yellow lightbulb icon with a blue base and radiating lines, symbolizing an idea or insight.

A module should contain at least 3-4 resources that logically go together.



So for the subnet folder we will grab 3 resources

- resource "aws_subnet"
- resource "aws_internet_gateway"
- resource "aws_default_route_table"



So I moved them:

```
terraform modules subnet main.tf
1 provider "aws" {
2   region = "eu-west-2"
3 }
4
5 resource "aws_vpc" "myapp-vpc" {...}
11
12 resource "aws_default_security_group" "default-sg" {
13   # name = "myapp-sg"
14   vpc_id = aws_vpc.myapp-vpc.id
15
16   ingress {
17     from_port = 22
18     to_port = 22
19     protocol = "TCP"
20     cidr_blocks = [var.my_ip]
21   }
22
23   ingress {
1 modules/.main.tf
1 resource "aws_subnet" "myapp-subnet-1" {
2   vpc_id = aws_vpc.myapp-vpc.id
3   cidr_block = var.subnet_cidr_block
4   availability_zone = var.avail_zone
5   tags = {
6     Name: "${var.env_prefix}-subnet-1"
7   }
8 }
9
10 resource "aws_internet_gateway" "myapp-igw" {
11   vpc_id = aws_vpc.myapp-vpc.id
12   tags = {
13     Name: "${var.env_prefix}-igw"
14   }
15 }
16
17 resource "aws_default_route_table" "main-rtb" {...}
```

Now we refactor the code to use variables (var.) instead of the object, see lines 2 and 11

```
1 resource "aws_subnet" "myapp-subnet-1" {
2   vpc_id = var.vpc_id
3   cidr_block = var.subnet_cidr_block
4   availability_zone = var.avail_zone
5   tags = {
6     Name: "${var.env_prefix}-subnet-1"
7   }
8 }
9
10 resource "aws_internet_gateway" "myapp-igw" {
11   vpc_id = var.vpc_id
12   tags = {
13     Name: "${var.env_prefix}-igw"
14   }
15 }
```

Same for line 18 below:

```

17 resource "aws_default_route_table" "main-rtb" {
18     default_route_table_id = var.default_route_table_id
19
20     route {
21         cidr_block = "0.0.0.0/0"
22         gateway_id = aws_internet_gateway.myapp-igw.id
23     }
24     tags = {
25         Name: "${var.env_prefix}-main-rtb"
26     }
27 }

```

Line 22 doesn't need the var. because the resource 'aws_internet_gateway' is created inside the current module -> so it is available.

The other ones were replaced because we don't have the vpc reference inside the module, it is outside in the root 'main.tf' so the vpc output will be passed as variables to the 'subnet' module.

So now we need to add the variables to the variables.tf of the 'subnet' module. Some variables already exist so we copy those from the root 'variables.tf'

```

1 variable vpc_cidr_block {}
2 variable subnet_cidr_block {}
3 variable avail_zone {}
4 variable env_prefix {}
5 variable my_ip {}
6 variable instance_type {}
7 variable "public_key_location" {}
8 variable "private_key_location" {}

```

And then we add the new variables that we created in the 'subnet' module main.tf

```
1 variable vpc_cidr_block {}
2 variable subnet_cidr_block {}
3 variable avail_zone {}
4 variable env_prefix {}
5 variable my_ip {}
6 variable instance_type {}
7 variable "public_key_location" {}
8 variable "private_key_location" {}

1 resource "aws_subnet" "myapp-subnet-1" {
2   vpc_id = var.vpc_id
3   cidr_block = var.subnet_cidr_block
4   availability_zone = var.avail_zone
5   tags = {
6     Name = "${var.env_prefix}-subnet-1"
7   }
8 }

10 resource "aws_internet_gateway" "myapp-igw"
11   vpc_id = var.vpc_id
12   tags = {
13     Name = "${var.env_prefix}-igw"
14   }
15 }

17 resource "aws_default_route_table" "main-rtb"
18   default_route_table_id = var.default_route_table_id

20   route {
21     cidr_block = "0.0.0.0/0"
22     gateway_id = aws_internet_gateway.myapp-
23   }
24   tags = {
25     Name = "${var.env_prefix}-main-rtb"
26 }
```

And we remove the variable 'subnet_cidr_block' from the root 'variables.tf' because we only need it for the subnet module:

```
1 variable vpc_cidr_block {}
2 #variable subnet_cidr_block {}
3 variable avail_zone {}
4 variable env_prefix {}
5 variable my_ip {}
6 variable instance_type {}
7 variable "public_key_location" {}
8 variable "private_key_location" {}

1 variable subnet_cidr_block {}
2 variable avail_zone {}
3 variable env_prefix {}
4 variable vpc_id {}
5 variable default_route_table_id {}
```

Use the Module

How do we reference the new module from another configuration file?

Using the keyword 'module'

So I will reference the module 'subnet' in the root 'main.tf' file


```
3 }
4
5 resource "aws_vpc" "myapp-vpc" {
6     cidr_block = var.vpc_cidr_block
7     tags = {
8         Name: "${var.env_prefix}-vpc"
9     }
10 }
11
12 module "myapp-subnet" {
13
14 }
15
16 resource "aws_default_security_group" "default-sg" {
17     # name = "myapp-sg"
18     vpc_id = aws_vpc.myapp-vpc.id
19 }
```

Now I pass the source of the module

```
11
12 module "myapp-subnet" {
13     source = "modules/subnet"
14 }
15
```

Now we pass the variables that we defined in 'subnet/variables.tf'

```
9 }
10
11
12 module "myapp-subnet" {
13     source = "modules/subnet"
14     subnet_cidr_block =
15 }
```

```
1 variable subnet_cidr_block {}
2 variable avail_zone {}
3 variable env_prefix {}
4 variable vpc_id {}
5 variable default_route_table_id {}
6
```

And we assign then values like we did with root 'terraform.tfvars' file and hardcode the value

```
11 module "myapp-subnet" {
12   source = "modules/subnet"
13   subnet_cidr_block = "10.0.10.0/24"
14 }
15
16 # resource "aws_default_security_group" "default-sg" {
17   name = "myapp-sg"
18   vpc_id = aws_vpc.myapp-vpc.id
19 }
20
21 ingress {
```

```
1 vpc_cidr_block = "10.0.0.0/16"
2 subnet_cidr_block = "10.0.10.0/24"
3 avail_zone = "eu-west-2a"
4 env_prefix = "dev"
5 my_ip = "140.228.47.252/32"
6 instance_type = "t2.micro"
7 public_key_location = "/Users/astrid/.ssh/"
8 private_key_location = "/Users/astrid/.ssh/"
```

```
1 variable subnet_cidr_block {}
2 variable avail_zone {}
3 variable env_prefix {}
4 variable vpc_id {}
5 variable default_route_table_id {}
```

Or we can reuse the 'terraform.tfvars' file to pass the value and use 'var.subnet_cidr_block' (see line 14 below in root main.tf)

```
11 module "myapp-subnet" {
12   source = "modules/subnet"
13   subnet_cidr_block = var.subnet_cidr_block
14 }
15
16 # resource "aws_default_security_group" "default-sg" {
17   name = "myapp-sg"
18 }
19
```

```
1 variable vpc_cidr_block {}
2 variable avail_zone {}
3 variable env_prefix {}
4 variable my_ip {}
5 variable instance_type {}
6 variable "public_key_location" {}
7 variable "private_key_location" {}
8
```

```
1 vpc_cidr_block = "10.0.0.0/16"
2 subnet_cidr_block = "10.0.10.0/24"
3 avail_zone = "eu-west-2a"
4 env_prefix = "dev"
5 my_ip = "140.228.47.252/32"
6 instance_type = "t2.micro"
7 public_key_location = "/Users/astrid/.ssh/"
8 private_key_location = "/Users/astrid/.ssh/"
```

```
1 variable subnet_cidr_block {}
2 variable avail_zone {}
3 variable env_prefix {}
4 variable vpc_id {}
5 variable default_route_table_id {}
```

But now we have a problem, the root main.tf file in line 14 will look at the root variables.tf file and we deleted the variable from there so we need to put it back:

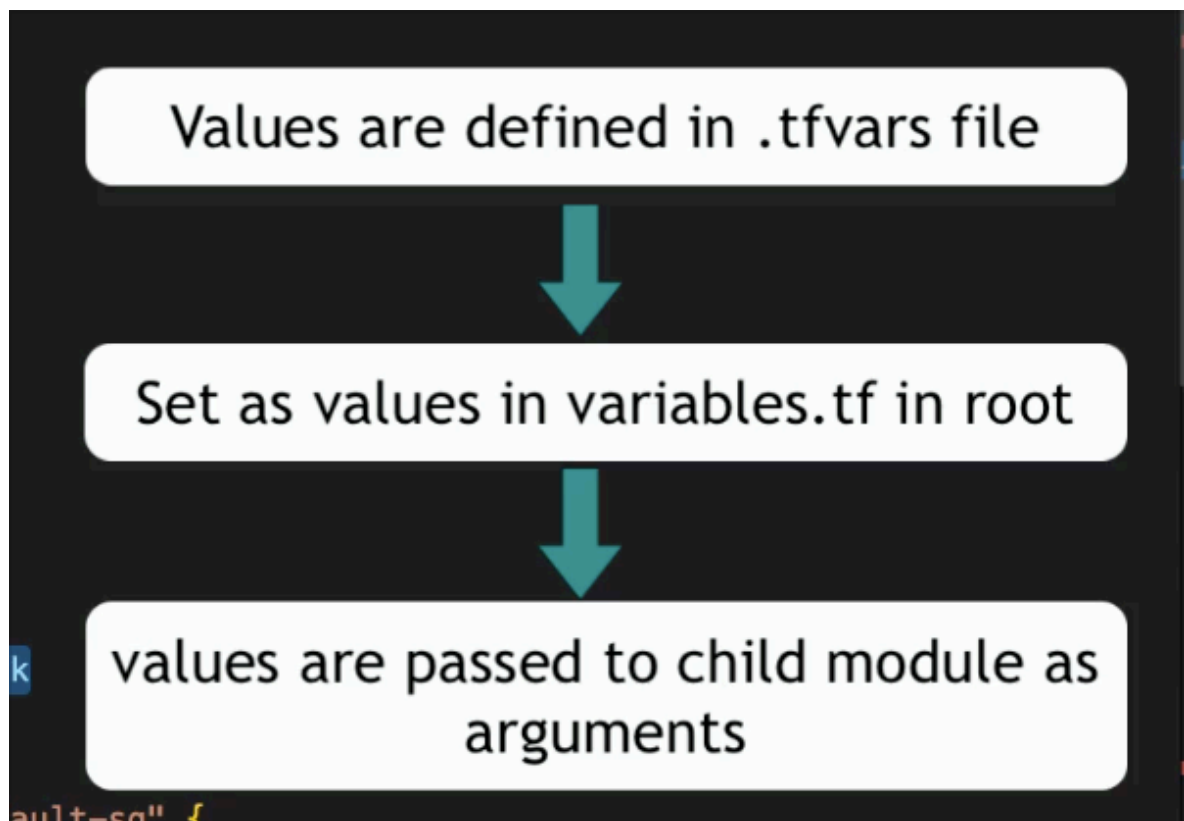
```
11 module "myapp-subnet" {
12   source = "modules/subnet"
13   subnet_cidr_block = var.subnet_cidr_block
14 }
15
16 # resource "aws_default_security_group" "default-sg" {
17   name = "myapp-sg"
18 }
19
```

```
1 variable vpc_cidr_block {}
2 variable subnet_cidr_block {}
3 variable avail_zone {}
4 variable env_prefix {}
5 variable my_ip {}
6 variable instance_type {}
7 variable "public_key_location" {}
8 variable "private_key_location" {}
9
```

```
1 vpc_cidr_block = "10.0.0.0/16"
2 subnet_cidr_block = "10.0.10.0/24"
3 avail_zone = "eu-west-2a"
4 env_prefix = "dev"
5 my_ip = "140.228.47.252/32"
6 instance_type = "t2.micro"
7 public_key_location = "/Users/astrid/.ssh/"
8 private_key_location = "/Users/astrid/.ssh/"
```

```
1 variable subnet_cidr_block {}
2 variable avail_zone {}
3 variable env_prefix {}
4 variable vpc_id {}
5 variable default_route_table_id {}
```

And now line 14 is not red



We add the other 2 variables 'avail_zone' and 'env_prefix'

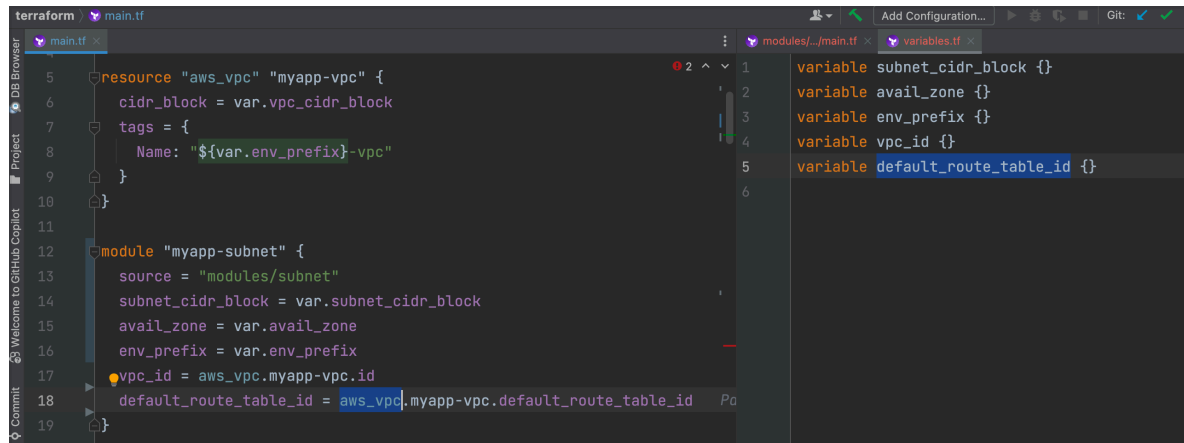
The screenshot shows the Terraform code editor with three files open: `main.tf`, `variables.tf`, and `modules/subnet/variables.tf`. In `main.tf`, a module `"myapp-subnet"` is defined with arguments `subnet_cidr_block`, `avail_zone`, `env_prefix`, and `aws_vpc.myapp-vpc.id`. In `variables.tf`, variables `vpc_cidr_block`, `subnet_cidr_block`, `avail_zone`, `env_prefix`, `my_ip`, `instance_type`, `public_key_location`, and `private_key_location` are defined. In `modules/subnet/variables.tf`, variables `subnet_cidr_block`, `avail_zone`, `env_prefix`, `vpc_id`, and `default_route_table_id` are defined.

Now we add 'vpc_id' and the value is not in 'terraform.tfvars' so we grab it from the root's main.tf itself as the vpc is created there so we have direct access

The screenshot shows the Terraform code editor with the same three files. In `main.tf`, the `resource "aws_vpc" "myapp-vpc"` is defined with `cidr_block = var.vpc_cidr_block` and `tags = { Name: "${var.env_prefix}-vpc" }`. In `variables.tf`, the variables are the same as in the previous screenshot. In `modules/subnet/variables.tf`, the variable `vpc_id` is now defined as `variable vpc_id {}`.

And the last variable we need to pass to our 'subnet' module is 'default_route_table_id' which also comes from the vpc object created in the

main.tf itself, so we have direct access to the default route table id:



We need to fix the source value, the relative path:

From "modules/subnet" to "./modules/subnet"

In Terraform:

- "./modules/subnet" — explicit relative path (relative to the main.tf file), starts from the current file's directory. Always a local path.
- "modules/subnet" — without ./, Terraform interprets it the same way for local modules, but the ./ is required for local modules to distinguish them from a registry module named modules/subnet.

Always use ./ for local modules. Without it, Terraform may try to look it up in the Terraform Registry instead of your local filesystem.

```
# correct - local module
module "subnet" {
  source = "./modules/subnet"
}
```

```
# wrong - Terraform will try to fetch "modules/subnet" from the registry
module "subnet" {
  source = "modules/subnet"
}
```

```

11
12 module "myapp-subnet" {
13     source = "../modules/subnet" You, 6 minutes ago • Uncommitted change
14     subnet_cidr_block = var.subnet_cidr_block
15     avail_zone = var.avail_zone
16     env_prefix = var.env_prefix
17     vpc_id = aws_vpc.myapp-vpc.id
18     default_route_table_id = aws_vpc.myapp-vpc.default_route_table_id
19 }

```

Now the creation of our EC2 instance complains in main.tf see line 81 📌

```

terraform > main.tf
main.tf x
75
76
77 resource "aws_instance" "myapp-server" {
78     ami = data.aws_ssm_parameter.latest-amazon-linux-image.value
79     instance_type = var.instance_type
80
81     subnet_id = aws_subnet.myapp-subnet-1.id
82     vpc_security_group_ids = [aws_default_security_group.default-sg.id]
83     availability_zone = var.avail_zone
84
85     associate_public_ip_address = true
86     key_name = aws_key_pair.ssh-key.key_name
87
88     user_data = file("entry-script.sh") // commented out to use provisioner
89
90     user_data_replace_on_change = true
91
92     tags = {
93         Name = "${var.env_prefix}-server"
94     }
95 }

```

Our subnet is now outside root main.tf in its own module 'subnet' -> so we need to fix the reference

Module Output

```
50     values = ["hvm"]
51   }
52 }
53
54 resource "aws_key_pair" "ssh-key" {
55   key_name = "server-key"
56   public_key = file(var.public_key_location)
57 }
58
59 resource "aws_instance" "myapp-server" {
60   ami = data.aws_ami.latest-amazon-linux-image.id
61   instance_type = var.instance_type
```



How do we access the resources of a child module?

How do we access resources of a child module?

1. We need the submodule to Output values -> like 'return'

Output Values

- like return value of module
- to expose/export resource attributes to parent module



So we create 'output' and place them in the outputs.tf

```
main.tf:73 key_name = "server-key"
main.tf:74 public_key = file(var.public_key_location)
main.tf:75
main.tf:76
main.tf:77 resource "aws_instance" "myapp-server" {
main.tf:78   ami = data.aws_ssm_parameter.latest-amazon-l
main.tf:79   instance_type = var.instance_type
main.tf:80
main.tf:81   subnet_id = aws_subnet.myapp-subnet-1.id
main.tf:82   vpc_security_group_ids = [aws_default_securi
main.tf:83   availability_zone = var.avail_zone
main.tf:84
modules/.main.tf:1 resource "aws_subnet" "myapp-subnet" {
modules/.main.tf:2   vpc_id = var.vpc_id
modules/.main.tf:3   cidr_block = var.subnet_cidr_block
modules/.main.tf:4   availability_zone = var.avail_zone
modules/.main.tf:5   tags = {
modules/.main.tf:6     Name = "${var.env_prefix}-subnet-1"
modules/.main.tf:7   }
modules/.main.tf:8 }
modules/.main.tf:9
modules/.main.tf:10 resource "aws_internet_gateway" "myapp
modules/.main.tf:11   vpc_id = var.vpc_id
modules/.main.tf:12   tags = {
outputs.tf:1 output "subnet" {
outputs.tf:2   value = aws_subnet.myapp-subnet-1
outputs.tf:3 }
```

The 'subnet' module output returns the whole subnet object, so now we can use the returned object as reference in our root main.tf and get any attribute of the object. The name of the output is 'subnet' and I will use that output name to make the reference:

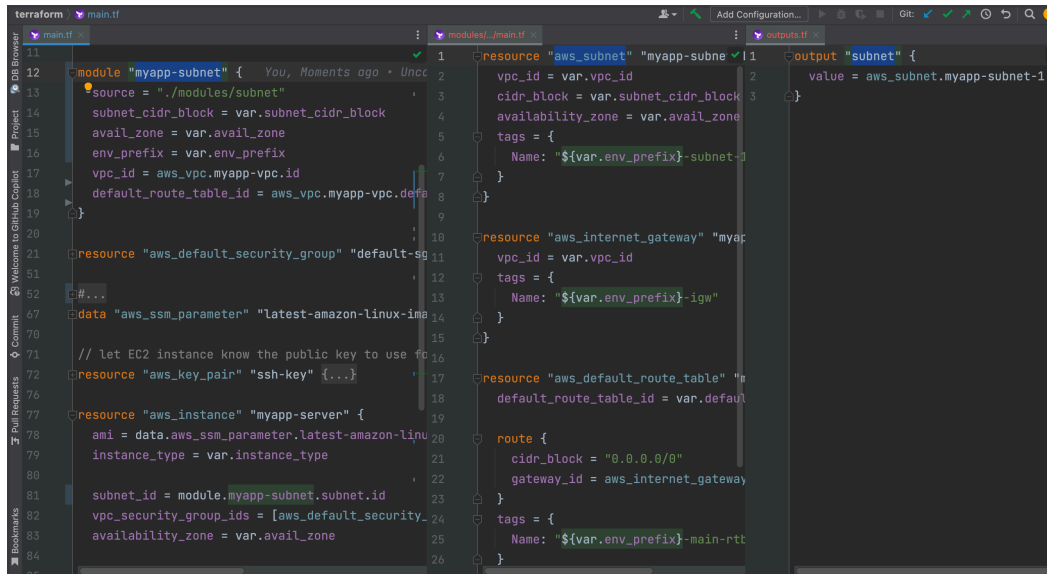
This is the syntax ->

module.<module_name>.<output_name>.<object_attribute>

-> module.myapp-subnet.subnet.id

myapp-subnet -> line 12 in root main.tf

subnet -> output name defined in subnet/outputs.tf



```
11 module "myapp-subnet" {
12   source = "../modules/subnet"
13   subnet_cidr_block = var.subnet_cidr_block
14   avail_zone = var.avail_zone
15   env_prefix = var.env_prefix
16   vpc_id = aws_vpc.myapp-vpc.id
17   default_route_table_id = aws_vpc.myapp-vpc.default_route_table_id
18 }
19
20 resource "aws_default_security_group" "default-sg" {
21   vpc_id = aws_vpc.myapp-vpc.id
22 }
23
24 data "aws_ssm_parameter" "latest-amazon-linux-image" {
25   name = "/aws/service/ami-amazon-linux-latest/amzn2-ami-linux-x86_64-gp2"
26 }
27
28 // let EC2 instance know the public key to use for ssh
29 resource "aws_key_pair" "ssh-key" {
30   name = "ssh-key"
31   public_key = data.aws_ssm_parameter.latest-amazon-linux-image.value
32 }
33
34 resource "aws_instance" "myapp-server" {
35   ami = data.aws_ssm_parameter.latest-amazon-linux-image.value
36   instance_type = var.instance_type
37   subnet_id = module.myapp-subnet.subnet.id
38   vpc_security_group_ids = [aws_default_security_group.default-sg.id]
39   availability_zone = var.avail_zone
40 }
41
42 resource "aws_subnet" "myapp-subnet" {
43   vpc_id = var.vpc_id
44   cidr_block = var.subnet_cidr_block
45   availability_zone = var.avail_zone
46   tags = {
47     Name = "${var.env_prefix}-subnet-1"
48   }
49 }
50
51 resource "aws_internet_gateway" "myapp-igw" {
52   vpc_id = var.vpc_id
53   tags = {
54     Name = "${var.env_prefix}-igw"
55   }
56 }
57
58 resource "aws_default_route_table" "myapp-default-rt" {
59   vpc_id = var.vpc_id
60   default_route_table_id = var.default_route_table_id
61 }
62
63 route {
64   cidr_block = "0.0.0.0/0"
65   gateway_id = aws_internet_gateway.myapp-igw.id
66 }
67
68 tags = {
69   Name = "${var.env_prefix}-main-rt"
70 }
```

Apply configuration changes

As we added a new module we need to execute (same if we make changes to a module)

-> terraform init

terraform init - whenever module added/changed

```
Terminal: zsh + v
) terraform init
Initializing modules...2/Terraform_chapter-17 +2 !3 ?1 terraform init
- myapp-subnet in modules/subnet
Initializing provider plugins found in the configuration...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.50.0

Initializing the backend...

Initializing provider plugins found in the state...

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

~/terraform Module_12/Terraform_chapter-17 +2 !3 ?1
```

We can see that there is an initialisation of modules, and we only have one initialised module called "myapp-subnet" which is located in modules/subnet
-> myapp-subnet in modules/subnet

We have not created the web server module yet, is not in the root main.tf

Let's preview the changes
-> terraform plan

```
# module.myapp-subnet.aws_default_route_table.main-rtb will be created
+ resource "aws_default_route_table" "main-rtb" {
  + arn                = (known after apply)
  + default_route_table_id = (known after apply)
  + id                 = (known after apply)
  + owner_id           = (known after apply)
  + region              = "eu-west-2"
  + route               = [
    + {
      + cidr_block      = "0.0.0.0/0"
      + gateway_id       = (known after apply)
      # (10 unchanged attributes hidden)
    },
  ]
  + tags                = {
    + "Name" = "dev-main-rtb"
  }
  + tags_all             = {
    + "Name" = "dev-main-rtb"
  }
  + vpc_id               = (known after apply)
}

# module.myapp-subnet.aws_internet_gateway.myapp-igw will be created
+ resource "aws_internet_gateway" "myapp-igw" {
```



```
# module.myapp-subnet.aws_subnet.myapp-subnet-1 will be created
+ resource "aws_subnet" "myapp-subnet-1" {
  + arn                                     = (known after apply)
  + assign_ipv6_address_on_creation        = false
  + availability_zone                      = "eu-west-2a"
  + availability_zone_id                   = (known after apply)
  + cidr_block                             = "10.0.10.0/24"
  + enable_dns64                           = false
  + enable_resource_name_dns_a_record_on_launch = false
  + enable_resource_name_dns_aaaa_record_on_launch = false
  + id                                     = (known after apply)
  + ipv6_cidr_block                        = (known after apply)
  + ipv6_cidr_block_association_id         = (known after apply)
  + ipv6_native                            = false
  + map_public_ip_on_launch                = false
  + owner_id                              = (known after apply)
  + private_dns_hostname_type_on_launch    = (known after apply)
  + region                                 = "eu-west-2"
  + tags                                  = {
    + "Name" = "dev-subnet-1"
  }
  + tags_all                              = {
    + "Name" = "dev-subnet-1"
  }
  + vpc_id                                = (known after apply)
}
```

```
Terminal: zsh x + v
+ owner_id                                = (known after apply)
+ private_dns_hostname_type_on_launch     = (known after apply)
+ region                                  = "eu-west-2"
+ tags                                    = {
  + "Name" = "dev-subnet-1"
}
+ tags_all                              = {
  + "Name" = "dev-subnet-1"
}
+ vpc_id                                = (known after apply)
}

Plan: 7 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ ec2_public_ip = (known after apply)

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform
apply" now.

~/terraform Module_12/Terraform_chapter-17 +2 !3 ?1 7s 12:38:24
```

No errors so far.

-> terraform apply

```
Terminal: zsh x + v
aws_instance.myapp-server: Creating...
aws_instance.myapp-server: Still creating... [00m10s elapsed]
aws_instance.myapp-server: Still creating... [00m20s elapsed]
aws_instance.myapp-server: Creation complete after 22s [id=i-05b5063e38f189c07]

Apply complete! Resources: 7 added, 0 changed, 0 destroyed.

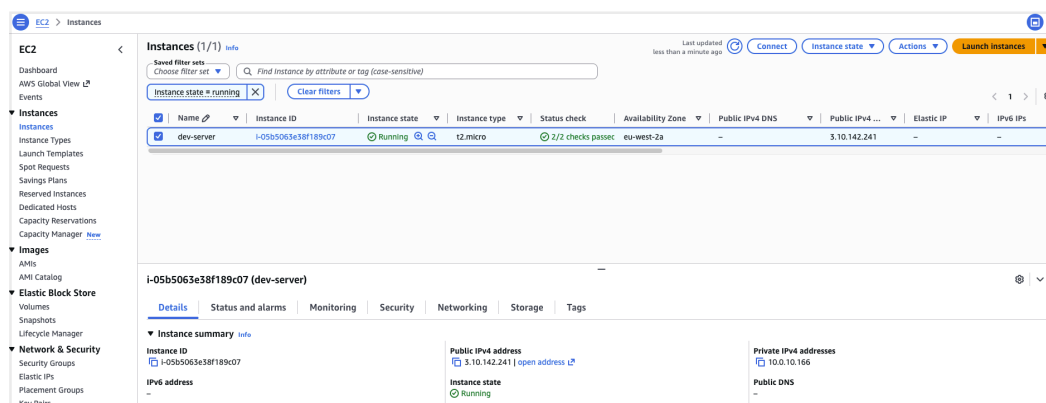
Outputs:

ec2_public_ip = "3.10.142.241"

~/terraform Module_12/Terraform_chapter-17 +2 !3 ?1
```

The output displayed in the terminal belongs to the root main.tf (main module), we don't see the submodule's outputs printed.

Let's check AWS UI



And it works 🥳